

Curs Algoritmi Fundamentali

Algoritmi Greedy

Tehnica alegerii unui optim local

Conf. dr. Alexandra Băicoianu

Universitatea *Transilvania* din Braşov
Facultatea de Matematică şi Informatică

2025

Agenda

- 1 **Prezentare generală**
- 2 **Verificarea corectitudinii**
- 3 **Diverse probleme**

Prezentare generală

- tehnica Greedy determină soluția unei probleme de optimizare pe baza alegerii unui optim local;
- alegerea se face (fără revenire) componentă cu componentă în soluția finală;
- folosirea criteriului de alegere se face pe setul de date de intrare, eventual cu o pregătire prealabilă acestora;
- modul de rezolvare al problemelor este construcția soluției componentă cu componentă, plecându-se inițial de la mulțimea vidă;
- dacă o componentă satisface criteriul de alegere corespunzător configurației curente, atunci participă la soluția finală;

Tehnica Greedy - Condiții

- o mulțime de **candidați**;
- o funcție care verifică dacă o anumită mulțime de candidați constituie o **soluție posibilă**, nu neapărat optimă, a problemei;
- o funcție care verifică dacă o mulțime de candidați este **fezabilă**, adică dacă este posibil să completăm această mulțime astfel încât să obținem o soluție posibilă, nu neapărat optimă, a problemei;
- o **funcție de selecție** care indică la orice moment care este cel mai promițător dintre candidații încă nefolosiți;
- o **funcție obiectiv** care dă valoarea unei soluții, funcția pe care urmărim să o optimizăm (minimizăm/maximizăm).

Algoritm Greedy - Descrierea formală

{C este mulțimea candidaților}

{S este mulțimea în care construim soluția}

subalgoritm Greedy(C) este:

$S \leftarrow \emptyset$

while not soluție(S) and $C \neq \emptyset$ do

$x \leftarrow$ un element din C care maximizează/minimizează

 select(x)

$C \leftarrow C - \{x\}$

 if fezabil($S \cup x$) then $S \leftarrow S \cup x$

if soluție(S) then returnează S

 else returnează "nu există soluție"

Algoritm Greedy - Exemplu

Se dă o fracție. Să se exprime ca sumă cu **număr minim** de fracții pozitive cu numărătorul 1.

Exemple: $\frac{6}{14}$ este $\frac{1}{3} + \frac{1}{11} + \frac{1}{231}$, $\frac{12}{13}$ este $\frac{1}{2} + \frac{1}{3} + \frac{1}{12} + \frac{1}{156}$.

Ce metodă de programare alegeți pentru rezolvare? De ce?

Este acest gen de exprimare unică? NU $\frac{5}{7} = \frac{1}{2} + \frac{1}{5} + \frac{1}{70} = \frac{1}{2} + \frac{1}{6} + \frac{1}{21}$

Algoritm Greedy - Observații (I)

- posibilul **inconvenient** al acestei tehnici constă în faptul că alegerea pe baza unui criteriu de optim local NU conduce întotdeauna spre un optim global
- metoda aleasă trebuie însoțită de **demonstrația** faptului că soluția obținută pe baza alegerii optimelor locale este în final un optim global
- algoritmi Greedy conduc în multe cazuri la soluții optime (și simplu de obținut), dar nu întotdeauna
- metoda Greedy este destul de puternică și se aplică în probleme din multe domenii (optimizare discretă, grafuri, probleme de transport, etc)

Algoritm Greedy - Observații (II)

- Soluția este construită pas cu pas ca la **Backtracking**, fără mecanismul de revenire la pasul anterior (ambele tehnici oferă soluții sub formă de vector, Greedy oferă (eventual) doar soluția optimă)
- Tehnica Greedy duce (de cele mai multe ori) la rezolvare în **timp polinomial**
- Pentru problemele pentru care nu se cunosc algoritmi care necesită timp polinomial, se caută soluții, chiar dacă nu optime, dar apropiate de acestea și care au fost obținute în timp util. Multe din aceste soluții sunt obținute cu Greedy. Astfel de algoritmi se numesc **algoritmi euristici**.

Verificarea corectitudinii

Multe dintre problemele pentru care soluția Greedy este optimă sunt caracterizate prin 2 proprietăți:

[1] Proprietatea **substructurii optime**: orice soluție optimă a problemei inițiale conține o soluție optimă a unei subprobleme (problema de același tip, dar de dimensiune mai redusă).

Când se poate spune despre o problemă că are proprietatea de substructura optimă? Atunci când pentru o soluție optimă $S = (s_1, \dots, s_k)$ a problemei de dimensiune n , subsetul $S(2) = (s_2, \dots, s_k)$ este o soluție optimă a unei subprobleme de dimensiune $n - 1$.

Demonstrația se face prin reducere la absurd.

Verificarea corectitudinii

[2] Proprietatea **alegerii Greedy**: componentele unei soluții optime au fost alese folosind criteriul Greedy de selecție sau pot fi înlocuite cu elemente alese folosind acest criteriu fără a altera proprietatea de optimalitate.

Când se poate spune că o problemă are proprietatea de alegere Greedy? Atunci când soluția optimă problemei fie este construită printr-o strategie greedy, fie poate fi transformată într-o altă soluție optimă construită pe baza strategiei greedy.

Minimizarea timpului mediu de așteptare (I)

Enunț: La o casă de marcat sunt serviți n clienți. Cunoscându-se timpul necesar de servire pentru fiecare client ($t_i, 1 \leq i \leq n$), să se determine o ordine de servire a clienților, astfel încât timpul mediu de așteptare să fie minim.

Exemplu: $t_1 = 5, t_2 = 10, t_3 = 3$. Câte posibile ordini de servire?

1	2	3	$5 + (5+10) + (5+10+3) = 38$	
1	3	2	$5 + (5+3) + (5+3+10) = 31$	
2	1	3	$10 + (10+5) + (10+5+3) = 43$	
2	3	1	$10 + (10+3) + (10+3+5) = 41$	
3	1	2	$3 + (3+5) + (3+5+10) = 29$	← optim
3	2	1	$3 + (3+10) + (3+10+5) = 34$	

Minimizarea timpului mediu de așteptare - Soluție (II)

Algoritmul de rezolvare de tip Greedy constă în strategia următoare:
la fiecare pas se selectează **clientul cu timpul minim de servire** din mulțimea de clienți rămasă.

```
std::sort(service_times.begin(), service_times.end());|

long long total_waiting_time = 0;
long long current_cumulative_service_time = 0;

for (int i = 0; i < n; ++i) {
    total_waiting_time += current_cumulative_service_time;
    current_cumulative_service_time += service_times[i];
}

double average_waiting_time = static_cast<double>(total_waiting_time) / n;

std::cout << "Timp total asteptare: " << total_waiting_time << std::endl;
std::cout << "Timp mediu asteptare: " << average_waiting_time << std::endl;
```

Minimizarea timpului mediu de așteptare - Corectitudine (III)

Fie $I = (i_1, i_2, \dots, i_n)$ o permutare oarecare a întregilor $\{1, 2, \dots, n\}$.

Dacă servirea are loc în ordinea I , avem:

$$T(I) = t_{i_1} + (t_{i_1} + t_{i_2}) + (t_{i_1} + t_{i_2} + t_{i_3}) + \dots = n * t_{i_1} + (n-1) * t_{i_2} + \dots = \sum_{k=1}^n (n-k+1) * t_{i_k}.$$

Pp că I este astfel încât putem avea doi întregi $a < b$ cu: $t_{i_a} > t_{i_b}$.

Interschimbam pe i_a cu i_b în I ; cu alte cuvinte, clientul care a fost servit al b -lea va fi servit acum al a -lea și invers.

Obținem o nouă ordine de servire J , care **este de preferat** deoarece:

$$T(J) = (n-a+1) * t_{i_b} + (n-b+1) * t_{i_a} + \sum_{k=1, k \neq a, b}^n (n-k+1) * t_{i_k}$$

$$T(I) - T(J) = (n-a+1) * (t_{i_a} - t_{i_b}) + (n-b+1) * (t_{i_b} - t_{i_a}) =$$

$(b-a)(t_{i_a} - t_{i_b}) > 0$. Deci, prin metoda Greedy obținem întotdeauna planificarea optimă a clienților.

Problema spectacolelor (I)

Enunț: Se dau n activități pentru care se cunosc momentele de început și de sfârșit.

- Orice activitate se desfășoară în intervalul $[s_i \leq t_i)$.
- Două activități sunt compatibile dacă duratele lor de desfășurare sunt disjuncte.

Se cere selectarea unei mulțimi *maximale* de activități compatibile între ele.

Problema spectacolelor - Soluție (II)

Algoritmul de rezolvare de tip Greedy constă în:

- Ordonăm spectacolele crescător după ora terminării lor.
- Selectăm inițial primul spectacol (deci cel care se termină cel mai devreme).
- Selectăm primul spectacol neselectat, care este disjunct față de cele deja selectate.
- Dacă nu se mai pot face alegeri, algoritmul se termină. Altfel, se selectează spectacolul găsit și se reia algoritmul de la pasul anterior.

Problema spectacolelor - Corectitudine (III)

Presupunem că setul de spectacole este ordonat crescător după momentul de terminare.

[1] **Proprietatea de alegere "greedy"**: Fie $O = (o_1, o_2, \dots, o_m)$ o soluție optimă și $X = (x_1, x_2, \dots, x_k)$ soluția dată de algoritm.

- ❶ Dacă $k > m$ atunci O nu este soluția optimă.
- ❷ Dacă $k = m$ atunci X este optimă.
- ❸ Dacă $k < m$, atunci putem înlocui în O pe o_1 cu x_1 (spectacolul care se termină cel mai repede) fără a altera restricția problemei și păstând același număr (maxim) de spectacole selectate.
Obținem soluția optimă $O' = (x_1, o_2, \dots, o_m)$.

Justificări suplimentare: Fie x_1 activitatea aleasă de algoritmul greedy (cea care se termină cel mai repede). Fie $O = (o_1, o_2, \dots, o_m)$ o soluție optimă arbitrară. Dacă $o_1 \neq x_1$, atunci, deoarece x_1 se termină cel mai devreme, avem $f(x_1) \leq f(o_1)$. Deoarece o_1 este compatibil cu o_2 (dacă există), rezultă că $f(o_1) \leq s(o_2)$ (momentul de început al activității o_2). Prin urmare, $f(x_1) \leq s(o_2)$, ceea ce înseamnă că x_1 este compatibil cu o_2 . Putem construi o nouă soluție $O' = (x_1, o_2, \dots, o_m)$ care este "la fel de optimă ca O " (având tot m activități) și care include alegerea greedy x_1 .

Problema spectacolelor - Corectitudine (III)

O soluție optimă la o problemă conține soluții optime la subproblemele sale.

[2] Proprietatea de substructură optimă: Considerăm soluția optimă $O' = (x_1, o_2, \dots, o_m)$. Presupunem că $(o_2, \dots, o_m)$ NU este soluție optimă pentru subproblema selecției subsetul considerat. Rezultă că există $O'' = (o_2'', \dots, o_{k''}'')$ o altă soluție cu $k'' > m$. Acest lucru ar conduce la o soluție $(x_1, o_2'', \dots, o_{k''}'')$ mai bună decât $O' = (x_1, o_2, \dots, o_m)$. Contradicție.

Justificări suplimentare: Demonstrat prin contradicție că, dacă subproblema $(o_2, \dots, o_m)$ nu ar fi optimă pentru activitățile rămase (cele care încep după $f(x_1)$), atunci am putea construi o soluție mai bună decât O' , ceea ce ar contrazice optimalitatea lui O' . Aceasta este exact modalitatea corectă de a demonstra substructura optimă.

Problema colorării unei hărți (I)

Enunț: Fiind dată o hartă cu n țări, se cere o soluție de colorare a hărții, utilizând cel mult 4 culori, astfel încât două țări care au frontieră comună să fie colorate diferit.

Exemplu:

(in)

6

alb verde galben rosu

1 2 1 3 1 4 1 5 1 6 2 3 2 5 2 6 3 4 3 5 4 5 5 6

(out)

1 alb 2 verde 3 galben 4 verde 5 rosu 6 galben

- Care este strategia optimă de rezolvare?
- Complexitate obținută?

Problema colorării unei hărți - Soluție (II)

Notății:

- matrice de vecini: $\text{matrice}[i][j]=1$ dacă țările i și j sunt vecine
- indCulTara - vector de indici a culorilor
- culori - vector de culori

Care este strategia optimă de rezolvare?

Avem un algoritm Greedy care va colora, pe rând, fiecare nod în cea mai mică (ca indice) culoare posibilă. Astfel, parcurgem țările și la fiecare pas în vectorul de indici al culorilor punem culoarea de prima poziție din vectorul de culori, dacă această culoare este întâlnită la o altă țară vecină atunci în vectorul de indici punem culoarea aflată pe a doua poziție în șirul de culori, procedeul repetându-se până se găsește culoarea potrivită.

Problema colorării unei hărți (III)

Inițializare: $\text{indCulTara} = [-1, -1, -1, -1, -1, -1]$ și culori = [alb, verde, galben, roșu]

Pas1: $\text{indCulTara} = [0, -1, -1, -1, -1, -1]$

Pas2: $\text{indCulTara} = [0, 0, -1, -1, -1, -1]$, $\text{indCulTara} = [0, 1, -1, -1, -1, -1]$

Pas3: $\text{indCulTara} = [0, 1, 0, -1, -1, -1]$, $\text{indCulTara} = [0, 1, 1, -1, -1, -1]$,
 $\text{indCulTara} = [0, 1, 2, -1, -1, -1]$

Pas4: $\text{indCulTara} = [0, 1, 2, 0, -1, -1]$, $\text{indCulTara} = [0, 1, 2, 1, -1, -1]$

Pas5: $\text{indCulTara} = [0, 1, 2, 1, 0, -1]$, $\text{indCulTara} = [0, 1, 2, 1, 1, -1]$,
 $\text{indCulTara} = [0, 1, 2, 1, 2, -1]$, $\text{indCulTara} = [0, 1, 2, 1, 3, -1]$

Pas6: $\text{indCulTara} = [0, 1, 2, 1, 3, 0]$, $\text{indCulTara} = [0, 1, 2, 1, 3, 1]$,
 $\text{indCulTara} = [0, 1, 2, 1, 3, 2]$

Problema colorării unei hărți (IV)

- Este demonstrat faptul că este nevoie numai de 4 culori pentru ca orice hartă să poată să fie colorată.
- NU se cunosc algoritmi polinomiali care să poată colora o hartă prin utilizarea a numai 4 culori.
- Dacă avem un număr mare de țări atunci suntem obligați să folosim o metoda euristică, în care obținem o colorare admisibilă, chiar dacă nu utilizeaza un număr minim de culori (poate chiar mai mare decât 4).
- Un algoritm *uristic*, bazat pe tehnica Greedy are avantajul unei complexități liniare (de cele mai multe ori), însă poate să nu utilizeze numărul minim de culori.
- Problema se poate rezolva cu backtracking (și obținem toate soluțiile), însă conduce către un algoritm exponențial.

Observații

- Un algoritm Greedy determină o soluție optimă a unei probleme pe baza unei succesiuni de alegeri
- Dacă strategia aleasă nu conduce la o soluție optimă atunci vorbim despre o **soluție Greedy euristică**.